UNIVERSIDAD POPULAR DEL CESAR

Programa de Ingeniería de Sistemas

REDA

Red de Evaluación y Detección de Riesgos Urbanos

Entrega 2 — Implementación

Curso: Base de Datos I

Integrantes: Rafael Alfonso Rivera Restrepo, Ivan Andrés Rodriguez Rodriguez, Luis Eliecer Mendoza Perpiñán, Carlos Eduardo Camelo Montaño

Valledupar, Cesar — 2026

1. Qué contiene esta entrega

Esta entrega implementa en PostgreSQL 18 el diseño de la Entrega 1, ampliado por un cambio de enfoque del negocio: REDA pasa a ser un **sistema de documentación probatoria de fallas en infraestructura pública**. El ciudadano reporta condiciones (antecedentes) e incidentes (daños sufridos); el sistema vincula cada incidente con los reportes previos del mismo punto ("la entidad ya sabía") y, si el caso es viable, lo remite a un abogado. El esquema pasa de 10 a 23 tablas, y se conservan las 10 originales con sus políticas de borrado.

Todo lo entregado está en el repositorio, en la etiqueta **entrega-2**, dentro de la carpeta **base-de-datos/** (las rutas de este documento son relativas a ella):

https://github.com/luismendoza009/REDA-Red-de-Evaluacion-y-Detecci-n-de-Riesgos-Urbanos

2. Dónde encontrar cada requisito

Requisito de la guía	Ubicación (dentro de base-de-datos/)	Resumen
1. Correcciones de la Entrega 1	`documentacion/entrega2/correcciones_entrega1.md`	Qué cambió en cada tabla y por qué
2. DDL como migraciones numeradas	`migraciones/001_ … 013_*.sql`	13 migraciones: PK, FK con política, NOT NULL, UNIQUE, CHECK, índices y vistas
Script de construcción	`scripts/migrar.ps1`	Base vacía → migraciones en orden (cada una en su transacción)
3. Datos semilla	`sql/semilla/*.sql`	137 reportes de Valledupar, 18 incidentes, 9 casos; simulados y coherentes
Generador de la semilla	`scripts/generar_semilla.py`	Determinista; documentado en documentacion/datos_simulados.md
4. Auditoría de normalización	`documentacion/entrega2/auditoria_normalizacion.md`	DF y forma normal de las 23 tablas, con 6 observaciones justificadas (E1–E6)
5. Banco de consultas	`sql/consultas/banco_consultas.sql`	10 consultas comentadas con la pregunta de negocio
Verificación de consultas	`sql/consultas/verificacion_consultas.sql`	Cada resultado recalculado por otro camino
6. Índices con evidencia	`documentacion/entrega2/evidencia_indices.md`	2 experimentos EXPLAIN ANALYZE antes/después

Salida de los planes	documentacion/entrega2/evidencia_indices.txt	Planes reales de PostgreSQL 18.6
7. Diccionario de datos	documentacion/diccionario_datos.md	Generado del esquema real con scripts/generar_diccionario.ps1
Diagrama EER actualizado	diagramas/reda_eer.drawio y .png	23 tablas, 31 relaciones, notación pata de gallo
Diagrama en notación de Chen	diagramas/reda_eer_chen.drawio y .png	Aporte de Rafael Rivera
Decisiones técnicas	documentacion/decisiones.md	Por qué de cada decisión y supuestos pendientes
Pruebas de restricciones	sql/pruebas/probar_restricciones.sql	32 operaciones inválidas que la base debe rechazar
Verificación desde cero	scripts/verificar_desde_cero.ps1	Base aparte: migraciones + semilla + todas las pruebas

3. Cobertura del banco de consultas

Exigencia de la guía	Consultas
2 consultas con JOIN de 3 o más tablas	C01 (6 tablas), C02, C05, C08
1 reunión externa "los que no tienen"	C03 (incidentes validados sin caso), C04 (condiciones sin foto)
1 agregación con GROUP BY y HAVING	C06 (zonas con 3+ fallas abiertas), C10
1 subconsulta o CTE	C05 (CTE, antecedentes con Haversine), C07 (subconsulta), C08 (CTE)
1 vista creada y consultada	C09 sobre v_resumen_casos, C10 sobre v_mapa_publico

4. Cómo se verificó

• **Migraciones sobre base vacía:** scripts/migrar.ps1 -Recrear crea la base desde cero y aplica las 13 migraciones sin errores; una segunda ejecución no reaplica ninguna. scripts/verificar_desde_cero.ps1 repite todo el proceso en una base aparte y corre las pruebas de esta sección.

• **Integridad declarativa:** las 32 pruebas negativas de sql/pruebas/probar_restricciones.sql son rechazadas, cada una por la restricción exacta que se esperaba (la prueba compara el nombre de la restricción).

• **Coherencia de la semilla:** las 15 comprobaciones de sql/pruebas/verificar_semilla.sql devuelven 0 (estado actual = último cambio del historial, hecho antes que el reporte, antecedentes dentro de 100 m, etc.).

• **Consultas:** las 10 verificaciones cruzadas coinciden (por ejemplo, LEFT JOIN … IS NULL = NOT EXISTS = EXCEPT; el total facturado por abogado = el total cobrado por caso). C02 y C07 se comprobaron además a mano.

• **Totales no inflados:** las vistas y consultas que combinan varias relaciones 1:N agregan en subconsultas antes de unir (ver v_resumen_casos y el comentario de C02).

5. Índices: resultados

Experimento	Antes	Después	Mejora
Búsqueda de antecedentes (índice compuesto categoría, tipo, latitud)	8,39 ms · 3 477 páginas	0,54 ms · 574 páginas	15× más rápido
Fotos del expediente (índice de la FK evidencias.id_reporte)	39,44 ms · Seq Scan de ≈ 400 000 filas	0,069 ms · 8 páginas	≈ 570× más rápido

Las mediciones se hicieron con 200 000 reportes y 400 000 evidencias sintéticos cargados dentro de una transacción que termina en ROLLBACK, para que el volumen no contamine la base de la demo.

6. Diagrama entidad-relación

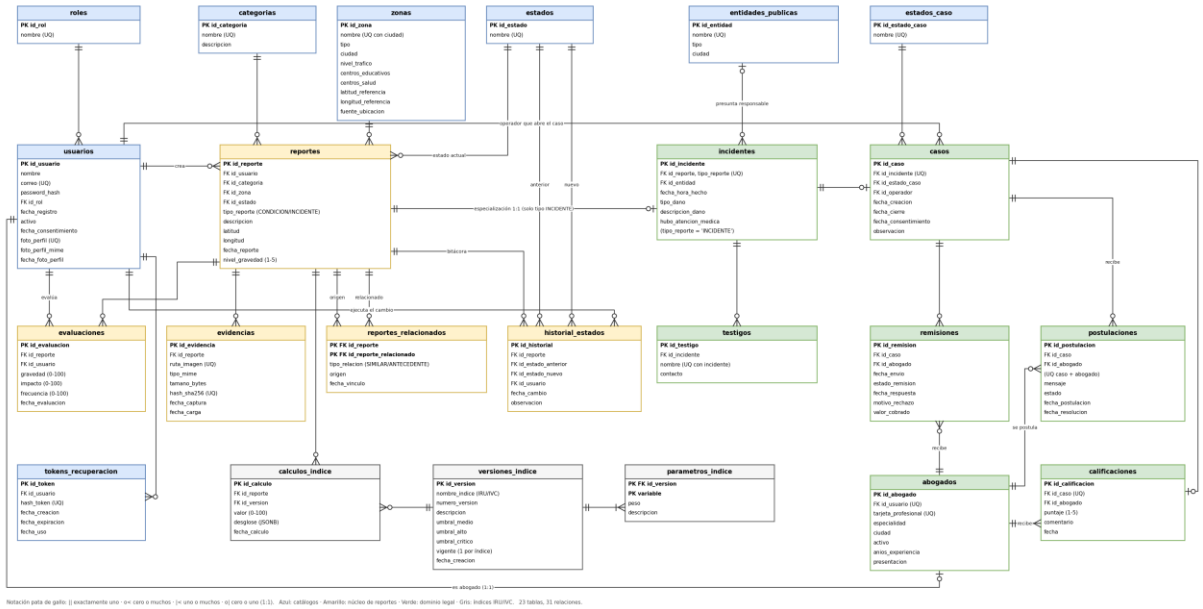


Figura 1. Modelo EER de REDA, Entrega 2. Archivo editable: diagramas/reda_eer.drawio.

7. Pendientes y riesgos declarados

• Reglas que las restricciones declarativas no alcanzan y que irán como triggers en la Entrega 3: que todo reporte INCIDENTE tenga su fila en incidentes, y que los pesos de cada versión de índice sumen 1.

• Funciones PL/pgSQL de cálculo de IRU e IVC y de búsqueda de antecedentes (Entrega 3).

• Los datos de la semilla son simulados (documentacion/datos_simulados.md). Los nombres de barrios y entidades están por verificar contra fuentes oficiales.

• Aspectos legales por validar con un abogado: modelo de cobro por caso remitido, plazo de caducidad de la acción de reparación directa y valor probatorio de fotos y geolocalización tomadas con una aplicación.

• Uso de inteligencia artificial: declarado en documentacion/declaracion_uso_ia.md.